Raport z etapu 5

Przetwarzanie dużych zbiorów danych

Szymon Jędrzejczak (266867)
Mateusz Nasewicz (268513)
Szymon Wieczorek (266586)

15.04.2026

Spis treści

1 Instalacja Hadoopa **2**
1.1 Skrypty uruchamiające . . . 2
1.2 Monitorowanie zadań YARN . . . 3
1.3 Weryfikacja wyników . . . 3
1.4 Replikacja danych . . . 4

2 Instalacja Spark on Hadoop **4**
2.1 Automatyzacja i konfiguracja środowiska . . . 4
2.2 Przetwarzanie danych w notatniku Spark . . . 5

1 Instalacja Hadoopa

Środowisko do przetwarzania danych zostało przygotowane w oparciu o architekturę rozproszoną, wykorzystującą Apache Hadoop. Klaster został wdrożony za pomocą skryptów automatyzujących konteneryzację Dockera (https://github.com/hjben/docker-server), co umożliwiło szybkie i powtarzalne zestawienie wymaganej infrastruktury.

Jak przedstawiono na poniższym zrzucie ekranu oraz w tabeli kontenerów, uruchomiono jeden węzeł master, zarządzający pracą klastra, oraz trzy węzły slave.

```
CONTAINER ID   NAMES     IMAGE                              STATUS
40ab40133489   master    hjben/hadoop-eco:3.3.0             Up 6 minutes
77c08467d55e   slave1    hjben/hbase:1.6.0-hadoop3.3.0      Up 6 minutes
317dc6c3d475   mariadb   hjben/mariadb:10.5                 Up 6 minutes
1612db1a40a2   slave3    hjben/hbase:1.6.0-hadoop3.3.0      Up 6 minutes
49a7fcbef69e   slave2    hjben/hbase:1.6.0-hadoop3.3.0      Up 6 minutes
```

Rysunek 1.1: Lista uruchomionych kontenerów klastra Hadoop

1.1 Skrypty uruchamiające

Automatyzacja procesu uruchamiania środowiska i wykonywania zadań została zrealizowana za pomocą skryptu bashowego. Skrypt ten wykonuje kilka kluczowych kroków: zatrzymanie poprzednich instancji, powołanie nowego klastra, oczekiwanie na gotowość HDFS, kopiowanie danych GTFS do kontenera oraz uruchomienie wybranych jobów.

```
#!/bin/bash

# Uruchamianie klastra
./compose-down.sh
./compose-up.sh 3.3.0 3 /tmp/hadoop /tmp/hadoop_logs ...

tmux has-session -t hadoop 2>/dev/null ||
tmux new-session -d -s hadoop './hadoop-start.sh start'

# Przygotowywanie danych
mvn clean package -DskipTests -q
cp "$JAR_PATH" "$DATA/"
docker cp $DATA $CONTAINER:$MASTER_DATA
docker exec "$CONTAINER" bash -c "
  set -e
  hdfs dfsadmin -safemode wait
  hdfs dfs -mkdir -p $HDFS_INPUT
  hdfs dfs -D dfs.replication=3 -put $MASTER_DATA $HDFS_INPUT/
"

# Przykladowe uruchomienie pierwszego etapu przetwarzania
yarn jar $MASTER_JAR $MAIN_CLASS \
      $HDFS_INPUT/data/gtfs/stop_times.txt \
      $HDFS_INPUT/data/gtfs \
      $HDFS_OUT_E11
```

1.2 Monitorowanie zadań YARN

Monitorowanie postępu i historii zadań można dokonać poprzez interfejs webowy YARN Resource Manager. Zadania przetwarzania danych GTFS (ekstrakcja, join oraz obliczenie wyniku) zakończyły się statusem SUCCEEDED.

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus
application_1776238868638_0002	root	E1.5 GTFS Stop Score	MAPREDUCE		default	0	Wed Apr 15 09:41:50 +0200 2026	Wed Apr 15 09:41:51 +0200 2026	Wed Apr 15 09:42:08 +0200 2026	FINISHED	SUCCEEDED
application_1776238868638_0001	root	E1.1 GTFS Extraction + Map Side Join	MAPREDUCE		default	0	Wed Apr 15 09:41:20 +0200 2026	Wed Apr 15 09:41:21 +0200 2026	Wed Apr 15 09:41:35 +0200 2026	FINISHED	SUCCEEDED

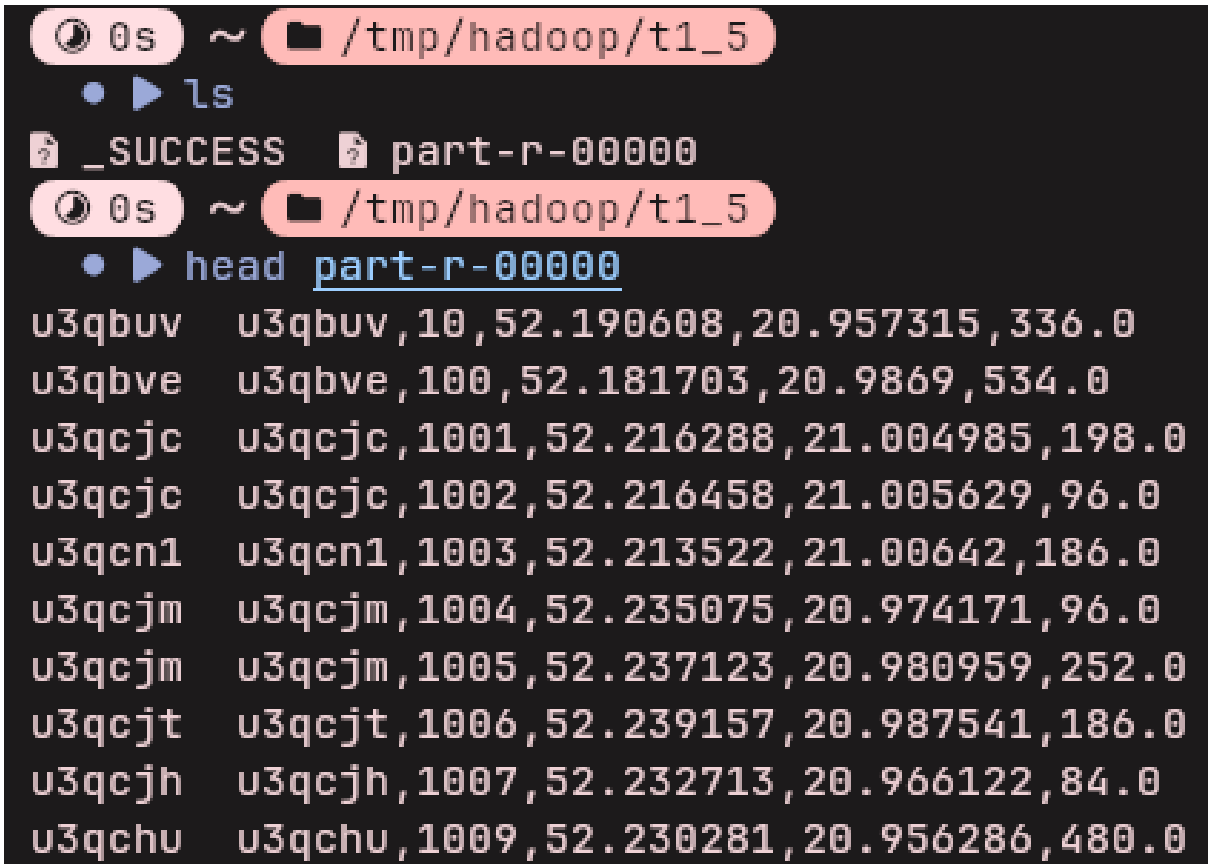
Rysunek 1.2: Zarządzanie zadaniami w interfejsie YARN

1.3 Weryfikacja wyników

Po zakończeniu zadań MapReduce weryfikację poprawności wygenerowanych plików umożliwia poniższa linijka kodu, która kopiuje dane z HDFS. Ścieżka na kontenerze */data/hadoop* jest zmapowana do lokalnego hosta, a dane są dostępne pod */tmp/hadoop*.

```
1 docker exec master bash -c "hdfs dfs -get /user/hadoop/t1_1 /data/hadoop/t1_1"
```

Poniżej przedstawiono przykładowe sprawdzenie wyników bezpośrednio z poziomu terminala oraz strukturę plików wynikowych.



Rysunek 1.3: Tymczasowy zbiór danych

1.4 Replikacja danych

Finalne dane są replikowane zgodnie ze zdefiniowaną polityką klastra (3 repliki).

☐	-rw-r--r--	root	supergroup	241 B	Apr 15 09:41	3	128 MB	agency.txt
☐	-rw-r--r--	root	supergroup	388 B	Apr 15 09:41	3	128 MB	calendar.txt
☐	-rw-r--r--	root	supergroup	35 B	Apr 15 09:41	3	128 MB	calendar_dates.txt
☐	-rw-r--r--	root	supergroup	198 B	Apr 15 09:41	3	128 MB	feed_info.txt
☐	-rw-r--r--	root	supergroup	1.4 MB	Apr 15 09:41	3	128 MB	routes.txt
☐	-rw-r--r--	root	supergroup	20.65 MB	Apr 15 09:41	3	128 MB	shapes.txt
☐	-rw-r--r--	root	supergroup	359.55 MB	Apr 15 09:41	3	128 MB	stop_times.txt
☐	-rw-r--r--	root	supergroup	347.31 KB	Apr 15 09:41	3	128 MB	stops.txt
☐	-rw-r--r--	root	supergroup	13.15 MB	Apr 15 09:41	3	128 MB	trips.txt

Rysunek 1.4: Replikacja zbioru źródłowego GTFS

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
☐	-rw-r--r--	root	supergroup	0 B	Apr 15 09:41	3	128 MB	_SUCCESS
☐	-rw-r--r--	root	supergroup	132.37 MB	Apr 15 09:41	3	128 MB	part-m-00000
☐	-rw-r--r--	root	supergroup	70.89 MB	Apr 15 09:41	3	128 MB	part-m-00001
☐	-rw-r--r--	root	supergroup	101.68 MB	Apr 15 09:41	3	128 MB	part-m-00002

Rysunek 1.5: Replikacja zbioru tymczasowego t1_1

2 Instalacja Spark on Hadoop

Kolejnym etapem prac było wdrożenie Apache Spark jako silnika do przetwarzania danych, działającego w oparciu o zasoby klastra Hadoop. Podobnie jak w przypadku bazowej instalacji, wykorzystano konteneryzację do powołania węzłów roboczych oraz dedykowanego środowiska Jupyter Lab, służącego jako interfejs programistyczny.

```
CONTAINER ID   NAMES         IMAGE                          STATUS
07817015d8c6   jupyter-lab   hjben/jupyter-lab:spark-3.4.0  Up 4 minutes
57ee4eb39e85   slave1        hjben/hadoop:3.3.5-jdk1.8.0    Up 4 minutes
5a60f6ddf8bd   spark         hjben/spark:3.4.0-livy         Up 4 minutes
6e1762e75562   slave2        hjben/hadoop:3.3.5-jdk1.8.0    Up 4 minutes
361b2d84cabe   slave3        hjben/hadoop:3.3.5-jdk1.8.0    Up 4 minutes
7f4d2e01f441   master        hjben/hadoop:3.3.5-jdk1.8.0    Up 4 minutes
```

Rysunek 2.1: Lista kontenerów klastra Spark on Hadoop

2.1 Automatyzacja i konfiguracja środowiska

W celu zapewnienia spójności środowiska wykonawczego przygotowano skrypt startowy.

Kluczowym elementem skryptu jest rozwiązanie problemu niezgodności wersji interpreterów Pythona pomiędzy Driverem a Workerami. Skrypt automatycznie instaluje i konfiguruje *ipykernel* oraz bibliotekę *pyspark* wewnątrz kontenerów, co zapewnia stabilną komunikację wewnątrz klastra YARN.

```
#!/bin/bash
set -e

WORKSPACE_DIR="$HOME/workspace/spark-notebook"
TADEUSZ=https://wolnelektury.pl/media/book/txt/pan-tadeusz.txt
CMD='./compose-up.sh 3.3.5 3.4.0 3 $HOME/workspace/spark-notebook /tmp/hadoop /tmp/hadoop_logs
↪  /tmp/spark_logs'
MASTER_DATA="/tmp/data"

# Przygotowanie zasobów lokalnych
mkdir -p "$WORKSPACE_DIR"
if [ ! -f "$WORKSPACE_DIR/pan_tadeusz.txt" ]; then
    wget -q -O "$WORKSPACE_DIR/pan_tadeusz.txt" $TADEUSZ
fi

if [ -f "words.ipynb" ]; then
    cp "words.ipynb" "$WORKSPACE_DIR/"
else
    echo "Błąd: Nie znaleziono notatnika."
    exit 1
fi

cd spark/docker-script/hadoop

# Uruchomienie klastra i oczekiwanie na gotowość HDFS
./compose-down.sh
tmux has-session -t spark 2>/dev/null || tmux new-session -d -s spark $CMD
while ! docker exec master hdfs dfsadmin -report > /dev/null 2>&1; do
  sleep 5
done

# Synchronizacja wersji Pythona i bibliotek
docker exec -u 0 jupyter-lab bash -c "
  curl https://bootstrap.pypa.io/get-pip.py | /usr/bin/python3.9
  /usr/bin/python3.9 -m pip install ipykernel
  /usr/bin/python3.9 -m ipykernel install --name python39 --display-name 'Python 3.9'
  /usr/bin/python3.9 -m pip install pyspark==3.4.0
"

# Inicjalizacja danych w systemie rozproszonym
docker exec master bash -c "hdfs dfsadmin -safemode wait"
docker cp $WORKSPACE_DIR master:$MASTER_DATA/
docker exec master hdfs dfs -D dfs.replication=3 -put $MASTER_DATA/pan_tadeusz.txt /
./jupyter-start.sh
```

2.2 Przetwarzanie danych w notatniku Spark

W celach testowych przeprowadzono analizę częstotliwości występowania słów na tekście utworu „Pan Tadeusz". Plik źródłowy został pobrany z zewnętrznego repozytorium, a następnie załadowany do HDFS z potrójną replikacją.

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	Apr 15 13:03	0	0 B	output
-rw-r--r--	root	supergroup	482.19 KB	Apr 15 13:02	3	128 MB	pan_tadeusz.txt
drwxrwx---	root	supergroup	0 B	Apr 15 13:02	0	0 B	tmp
drwxr-xr-x	root	supergroup	0 B	Apr 15 13:03	0	0 B	user

Rysunek 2.2: Weryfikacja replikacji pliku tekstowego na HDFS

Przetwarzanie zrealizowano, wykorzystując klaster Hadoop jako warstwę składowania danych. Plik wejściowy został pobrany bezpośrednio z systemu plików HDFS (protokół `hdfs://`), a po wykonaniu operacji zliczania słów wyniki zapisano z powrotem na HDFS w formie partycji tekstowych.

```
import os
os.environ['PYSPARK_PYTHON'] = '/usr/bin/python3.9'
os.environ['PYSPARK_DRIVER_PYTHON'] = '/usr/bin/python3.9'

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Word Count - Pan Tadeusz") \
    .master("yarn") \
    .config("spark.cores.max", "4") \
    .config("spark.sql.shuffle.partitions", "5") \
    .config("spark.executorEnv.PYSPARK_PYTHON", "/usr/bin/python3.9") \
    .config("spark.executor.extraJavaOptions", "-XX:-UseContainerSupport") \
    .config("spark.driver.extraJavaOptions", "-XX:-UseContainerSupport") \
    .getOrCreate()

input_data = "hdfs://master:9000/pan_tadeusz.txt"
output_data = "hdfs://master:9000/output/pan_tadeusz_count"

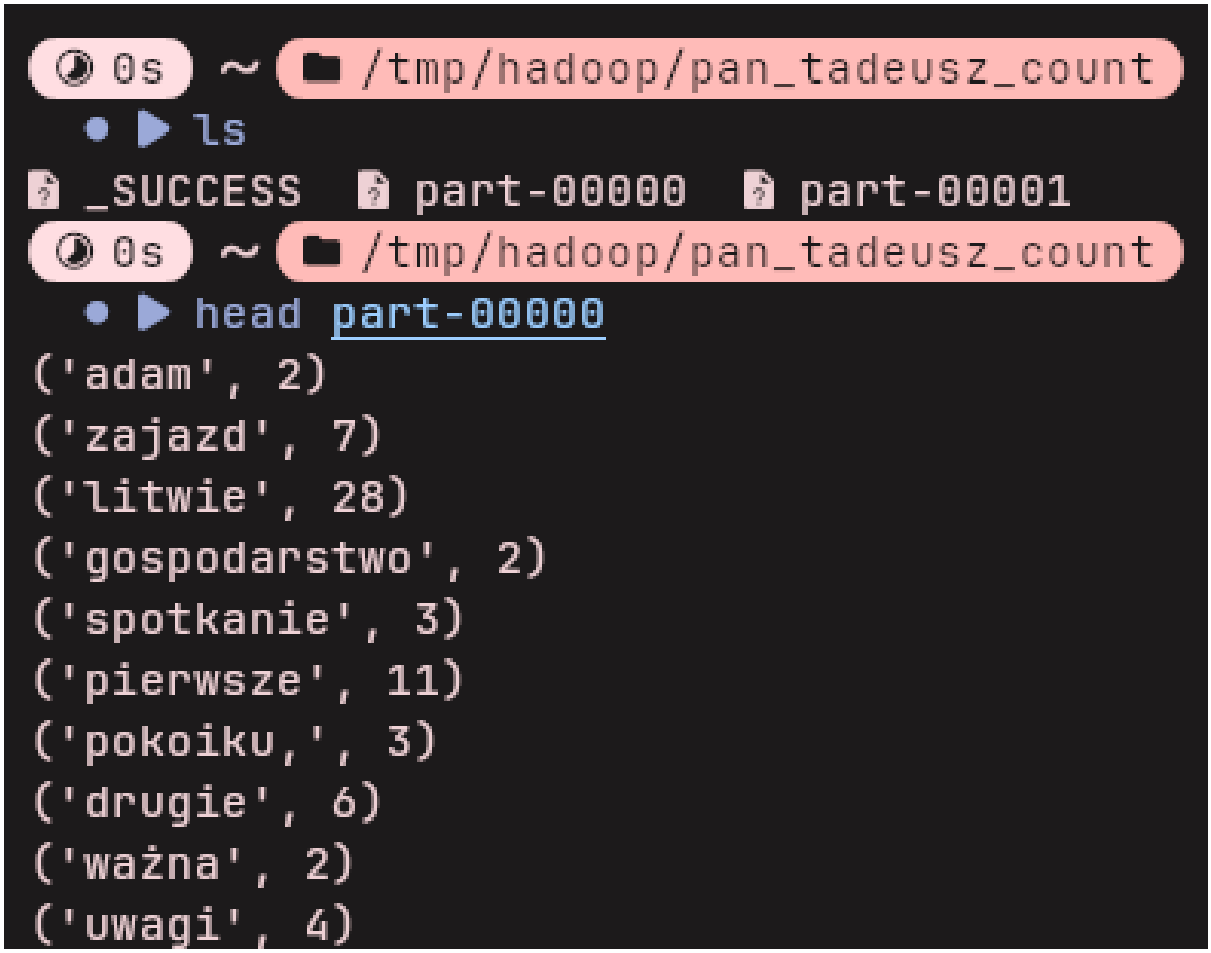
lines = spark.sparkContext.textFile(input_data)

out = lines.flatMap(lambda line: line.lower().split()) \
           .map(lambda word: (word, 1)) \
           .reduceByKey(lambda a, b: a + b)

out.saveAsTextFile(output_data)

print(f"Zadanie zakończone! Wyniki zapisano na HDFS: {output_data}")

spark.stop()
```



Rysunek 2.3: Struktura katalogu wyjściowego Spark po zakończeniu obliczeń

Finalne sprawdzenie zawartości katalogu wyjściowego potwierdziło poprawność działania algorytmu. Obecność pliku `_SUCCESS` świadczy o bezbłędnym zakończeniu zadania przez wszystkich executorów YARN.